Spoiler-Restricted Customized TV Recap Generation

Arion Tripathi
Boston University
arionct@bu.edu

Abstract

It is often hard to find a plot summary of a long TV show that lines up with exactly how far a viewer has watched and that does not give away later events. In this work, I build a local recap generator where the user gives a show and a stopping point k (season and episode), and the system generates a recap only up to that point. I compare three Gemma-backed methods: a no-retrieval baseline, retrieval over full episode transcripts, and retrieval over episode summaries. The retriever is deterministic rather than learned: it filters the data to episodes at or before k before chunking, so no later episode text is placed in the prompt. Since the midway report, I added a no-retrieval baseline, a summary-based corpus, timing logs, automated retrieval tests, and quantitative overlap metrics for grounding and spoiler risk. The main result is that summary RAG performs best overall. For an early episode, the baseline produced 7 future-only entity hits and major spoilers, while summary RAG produced 0 future-only entity hits and a much more useful recap. For a later episode, uncapped summary RAG had much stronger retrieved-source coverage than transcript RAG (0.715 vs. 0.278), though it still hallucinated some details. Overall, RAG helps most when the retrieved text is already summary-like, but spoiler-safe recap generation is still not solved because the model can use memorized show knowledge.

1 Introduction

It is common that a person will begin a TV show, take some time off from it, then pick up with it again later. In this case, they might have forgotten some major plot points, particularly if they are returning in the middle of a season as opposed to the beginning of a new season. This problem is nontrivial because, while popular shows may have per-episode summaries, there does not usually exist a collection of recaps of an entire show up until arbitrary episodes. A viewer who stopped at Season 3 Episode 13 likely does not want to read a collection of different summaries. They might want one coherent recap that compresses older context and highlights the important plot threads needed to continue watching.

The obvious solution is to ask a chatbot language model for a recap up to a given episode. This is risky because popular shows are likely present in the model's training data. The model may minimize the requested stopping point, use its memorized knowledge of the full show, or accidentally include foreshadowing that the viewer should not know yet. In this project, the task is therefore:

$$f(\text{show}, k) \rightarrow \text{recap}_{\leq k}$$

where k is a season and episode boundary, and the generated recap should only include information from episodes at or before k. The main goal is not just fluency. A polished recap with spoilers is worse than a rougher recap that stays within the viewer's watched context.

I focus on *Breaking Bad* because it is popular, has many public transcript resources, and is exactly the kind of show where spoilers are important. The final system compares three model-based versions: no retrieval, transcript RAG, and summary RAG. I briefly tested extractive fallback methods during development, but I removed them from the final comparison because this project is intended to study model-based recap generation rather than only sentence selection.

2 Related Work

This project is an application of retrieval-augmented generation. RAG systems retrieve relevant source passages and then condition a generator on those passages (Lewis et al., 2020). My setting is different from open question answering because the key constraint is not just relevance, but also chronology. A chunk about a future episode may

be highly relevant to the show, but it is invalid if it occurs after the user's stopping point.

I use Gemma 3 4B through Ollama's chat interface as the local generator. Gemma 3 is a family of lightweight open models with instruction-following variants and long context support (Gemma Team, 2025). I did not fine-tune the model. This makes the experiment closer to what a normal user might try locally, but it also means the model may rely on parametric memory of the show.

For evaluation, I draw on two summarization-evaluation ideas. BERTScore compares generated text to reference text using contextual embeddings rather than exact string overlap (Zhang et al., 2020). I did not use it as the main metric because this project does not have gold cumulative recaps for every stopping point. I also used the idea from Liu et al. (2022) that reference-free summary evaluation can compare a summary to its source document while accounting for compression. My implementation is only a lightweight version, described below, but it helped make evaluation more quantitative than previously.

3 Methods

3.1 Data

The first step was data collection. I decided to start with *Breaking Bad*, as it is one of the most popular TV shows and had multiple transcript and summary resources online. The search for transcript resources was nontrivial because many options were either incomplete or inconsistently formatted. I found a Kaggle dataset of *Breaking Bad* scripts with character names for each spoken line (Sial and mexwell, n.d.), but the data only went up to episode 6 of season 3. I found another resource with transcripts for the whole show, but character line labels were only available through the last episode of season 3 (Forever Dreaming Transcripts, n.d.). I chose a single consistent transcript source for the whole show (SubsLikeScript, n.d.), even though it did not include speaker labels.

The raw transcript corpus contains one text file per episode. I scraped the pages, removed subtitle clutter and metadata, and converted the data into JSON rows with show, season, episode, episode id, and text. The processed transcript file has all 62 episodes of the show. After the midway report, I added a second corpus of per-episode summaries from the Studying Breaking Bad Course site (Studying Breaking Bad Course, n.d.). These summaries are still episode-limited, but they are more plot-focused than raw transcripts. This matters because a full transcript contains jokes, repeated dialogue, scene directions, and fragments that are not always useful for a cumulative recap. Another limitation of the raw transcripts was the lack of character line tagging resulting in the data being an unclean collection of contextless words.

3.2 Retrieval

The retriever is deterministic and hand-coded. It is not BM25, DPR, dense retrieval, or a learned ranker. For a requested stopping point $k = (s, e)$, the code first filters the episode rows:

$$R_k = \{r : r.\text{show} = \text{show} \wedge (r.s, r.e) \leq (s, e)\}.$$

Only after this filter does the system build chunks. This ordering is important: future episodes are removed before chunking, so a chunk cannot accidentally cross the stopping boundary. For transcript RAG, episodes are split into fixed 400-word windows. For summary RAG, summaries are split by chapter headings that were integrated by the scraped website, with word-based chunking as a fallback.

When a context budget is needed, the default selector is episode-balanced. Earlier versions used a tail strategy, which kept the most recent chunks. That was reasonable for remembering the immediate plot, but it caused a mismatch: the prompt asked for a cumulative recap even when early episodes had been dropped from the context. The episode-balanced selector gives each included episode at least one chunk when possible, then allocates remaining chunks backward through the window. This still never introduces episodes after k.

I added unit tests that check the retrieval guarantee directly rather than only checking final recaps. Several tests build small fake episode lists with obvious banned future text. For example, one test requests Season 2 Episode 5 while adding a Season 2 Episode 6 row containing a unique forbidden token; after filtering and chunking, the test asserts that the forbidden token never appears in the retrieved chunks. Another test does the same for later seasons. A real-data test loads the processed `episodes.json`, requests Season 2 Episode 5, and verifies that every returned episode is at or before that boundary. Other tests check chunk-budget behavior, summary-corpus path resolution,

no-retrieval prompt formatting, cumulative prompt wording, and the single-episode guard that cuts off generated text if the model starts an invented later-episode heading. In total, the current system passes 36 tests. I also manually inspected the retrieved episode id lists in the JSON run logs for the representative outputs.

3.3 Generation Settings

All three final methods use the same local generator, `gemma3:4b`, served by Ollama through an OpenAI-compatible local endpoint. The temperature was 0.2 and `max_tokens` was 4096. The prompt tells the model to write one cohesive recap, not a per-episode index. This was an important change: the system should not produce three lines for every episode the user has watched, because a Season 5 request would become too long to be useful. The target length is 250–350 words for normal requests, with longer outputs allowed only for late-show stopping points where extra context is necessary.

The three final methods are:

- **No-retrieval baseline**: Gemma is prompted to generate a recap and receives only the show name and stopping point. This tests whether prompting alone can prevent spoilers.
- **Transcript RAG**: Gemma receives transcript chunks from episodes at or before the stopping point.
- **Summary RAG**: Gemma receives summary chunks from episodes at or before the stopping point.

The compare script runs these three methods in the same order and writes the outputs, metrics, timing, retrieved chunk counts, and included episode ids to JSON. In the representative runs, individual generations took about 30–60 seconds, so a three-method comparison usually took around two minutes.

3.4 Evaluation

I evaluated outputs both manually and with automatic metrics. The manual score is on a 0–10 scale where spoiler prevention and factual grounding matter more than style. A fluent but spoiled recap receives a low score.

The first automatic metric splits the corpus into three regions relative to k: episodes strictly before k, the stopping episode itself, and future episodes. I compute token overlap between the generated recap and each region, both with all tokens and with stopwords removed. This was motivated by the suggestion to check whether the output overlaps more with allowed episodes than with future episodes. In practice, raw future overlap is noisy because common show vocabulary appears throughout the series. For that reason, I treat it as a relatively weak contributor rather than the main score.

The more useful spoiler test is future-only hits. I extract simple capitalized phrases from the generated recap and check whether any of those phrases appear only in future episodes, not in prior or at-k text. This is not a perfect recognizer, so it can produce false positives such as ordinary capitalized words. However, it is helpful for catching obvious future-only names such as later characters. I use this because in the midway report the system had a tendency to discuss characters that had not yet been introduced, likely due to parameter memorization since I verified that future chunks were not being retrieved.

For grounding, I compute retrieved content coverage: the fraction of generated content tokens that also appear in the retrieved source text. I also compute unsupported content token ratio: the fraction of unique generated content tokens not found in the retrieved source. Lower unsupported ratio is likely better but could also just be the use of more creativity in generation. Finally, I implemented a lightweight Liu et al. (2022)-style metric: compression token ratio, character-bigram Jaccard overlap between the generation and retrieved text, and a compression-adjusted Jaccard score. This is not the full pretrained-language-model semantic distribution metric from the paper, but it gives a fast reference-free indicator for whether the recap is staying close to the retrieved source.

4 Experiments

I used two representative stopping points. The first was Season 1 Episode 3, an early point where spoilers are easy to detect because many famous characters and plot lines should not appear yet. The second was Season 3 Episode 13, a much later point where the system has to compress a much larger amount of context. This later test is harder because a useful recap must integrate many previous events without growing linearly with the number of episodes.

Method	Gen. toks	Fut. ent.	Fut.-only ratio	R. cov.	Unsup. ratio	Score
No retrieval	328	7	0.416	N/A	N/A	1.5
Transcript RAG	245	1	0.440	0.192	0.828	2.5
Summary RAG	298	0	0.358	0.343	0.689	6.5

Table 1: Results for *Breaking Bad* Season 1 Episode 3. "Fut. ent." is future-only entity hits. "Fut.-only ratio" is the ratio of generated content-token types that appear only in future episodes. "R. cov." is retrieved content coverage. "Unsup. ratio" is unsupported content token ratio. "Score" is qualitative heuristic for manual inspection of direct spoilers, excessive foreshadowing, and general recap quality (out of 10.0). Refer to `SELECTED_RUN_OUTPUTS.md` in the GitHub link to view the generated recaps.

Method	Gen. toks	Fut. ent.	Fut.-only ratio	R. cov.	Unsup. ratio	Score
No retrieval	340	2	0.068	N/A	N/A	1.5
Transcript RAG	255	0	0.117	0.278	0.775	3.0
Summary RAG	313	1	0.103	0.715	0.340	5.5

Table 2: Results for *Breaking Bad* Season 3 Episode 13.

5 Results and Discussion

The early episode results are the clearest. The no-retrieval baseline was fluent, but it failed the actual task. It introduced later characters and events, such as Tuco and Hector Salamanca, even though the requested stopping point was Season 1 Episode 3. The future-only entity count was 7, and manual inspection showed that several details were not just vague but directly wrong for that point in the show. This supports the main concern behind the project: prompting a memorized model to avoid spoilers is not enough.

Transcript RAG avoided the worst famous-character spoilers in the early run, but the recap was not very useful. It described vague investigations and chemical measurements without clearly explaining the main story. This suggests that the model was grounded in the transcript text, but the raw transcript chunks were too indirect and cluttered for a short cumulative recap. The retrieved content coverage was only 0.192 and the unsupported ratio was 0.828, which matches the qualitative impression that the model was not cleanly using the transcript chunks.

Summary RAG was the best early method. It had 0 future-only entity hits, higher retrieved content coverage than transcript RAG, and a lower unsupported ratio. The recap was not perfect: it still contained some unsupported phrasing and dramatized wording, and the lexical future overlap was still relatively high because many generic words occur throughout the series. However, it gave a more coherent recap of Walt's diagnosis, his partnership with Jesse, the early meth operation, and the Krazy-8 conflict without naming obviously future-only characters. This was the first setting where retrieval clearly beat the no-retrieval baseline both quantitatively and qualitatively.

The later episode results are more mixed. The no-retrieval baseline again failed manually because it used full-series knowledge. It discussed a death that had not occurred in the show yet and other events beyond the Season 3 Episode 13 boundary. Its future-only entity count was lower than in the early test, but that does not mean it was safe; after three seasons, many important names have already appeared, so entity overlap alone misses some future spoilers. This is why manual inspection remained necessary.

Transcript RAG improved spoiler safety in the later run, with 0 future-only entity hits, but it did not solve the recap task. Because only 8 transcript chunks were included, the prompt contained chunks from Season 3 Episode 6 through Season 3 Episode 13 and omitted earlier episodes. The output therefore focused narrowly on a small window around Jesse, Gale, Saul, and Mike instead of providing a real cumulative recap from the pilot through the stopping point. This was a good example of why a context cap must be aligned with the user's requested output. If the prompt asks for a cumulative recap but the retriever drops most early context, the model either omits important history or fills gaps from memory.

Uncapped summary RAG was better for the late setting but still imperfect. Quantitatively, it was the strongest grounded method: retrieved content coverage rose to 0.715 and unsupported ratio fell to 0.340, a large improvement over transcript RAG. Qualitatively, it integrated more of the series context and stayed closer to the desired 250–350 word range. However, it still produced some hallucinated

or warped claims, such as witness protection and unclear causal descriptions around Gale and Gus. This shows that giving the model more allowed context is not always enough. The generator can still compress incorrectly, especially when the prompt is long and the show is well known.

Overall, the main result is that summary RAG is the best of the three model-based approaches. Transcript RAG has a stronger claim to primary-source grounding, but it asks the model to infer a plot summary from messy unlabeled dialogue fragments. No-retrieval generation is the most fluent but the least trustworthy.

6 Progress Since Midway

First, I added the no-retrieval baseline, which made the comparison much clearer. The baseline showed that Gemma's memorized knowledge of *Breaking Bad* is a real source of spoilers and hallucinations, not just a possible concern.

Second, I made the deterministic retriever more explicit and better tested. The goal of this retriever is not to rank all show passages by relevance; it is to enforce the spoiler boundary structurally. Unit tests check that episodes after k are not retrieved.

Third, I added quantitative evaluation. The token-overlap metric by prior, at-k, and future regions was useful as an exploratory check, but it was too noisy to use alone. Future-only entity hits were more helpful for spoiler smoke testing. Retrieved content coverage and unsupported content ratio were more helpful for grounding. I also implemented a compression and source-overlap proxy inspired by Liu et al. (2022).

Fourth, I moved beyond transcript-only RAG. In the midway report, I suspected that line splitting and speaker changes might be the main problem. After more tests, I think the bigger problems are parametric memory and the difficulty of asking a small model to infer clean plot structure from raw transcripts. The summary corpus directly addresses that issue by giving the model cleaner source material while keeping the same episode boundary.

7 Limitations and Future Work

I hypothesize that a large limitation to the transcript retrieval system is the lack of comprehensive, labeled character lines. Since I was unable to find a source that provided such labeled data in entirety, I used unlabeled data which I suspect caused confusion for the model. It would be interesting to examine how labeling the data or using pre-labeled lines such as closed captions from a streaming service could impact the system's performance.

The second limitation is evaluation. My metrics are useful for comparing iterations, but they are not a complete spoiler detector. Token overlap with future episodes is noisy, and the simple extractor can miss spoilers that do not introduce new names. Thus, human manual evaluation remains the ideal method which is time consuming and not automatable.

The third limitation is scaling to arbitrary shows. The project currently depends on scraped resources for *Breaking Bad*. For a general system, the data pipeline would need to handle different transcript formats, missing episodes, inconsistent episode numbering, and possibly shows where no good public summaries exist. This is where the integration with a streaming service would also likely prove valuable as closed caption data along with episode metadata would all likely be uniformly formatted.

8 Conclusion

This project built and evaluated a spoiler-restricted recap generator for *Breaking Bad*. The final comparison keeps three model-based methods: no retrieval, transcript RAG, and summary RAG. The no-retrieval baseline was fluent but unsafe. Transcript RAG structurally avoided future episode text but often produced vague or incomplete recaps. Summary RAG was the best overall method, especially for grounding and early-episode spoiler prevention, but it still hallucinated some details in later settings. The main takeaway is that RAG helps, but the quality of the retrieved corpus matters as much as the retrieval boundary. A practical spoiler-safe recap system needs both episode-limited retrieval and stronger verification of generated claims.

9 Source

The source code is available at `https://github.com/arionct/Custom-TV-Recap-Generation`.

References

Forever Dreaming Transcripts. n.d. Breaking Bad. Episode transcripts. Accessed: 2026-04-14.

Gemma Team. 2025. Gemma 3 Technical Report. *arXiv preprint arXiv:2503.19786*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Hein-

rich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.

Yizhu Liu, Qi Jia, and Kenny Zhu. 2022. Reference-free Summarization Evaluation via Semantic Correlation and Compression Ratio. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2109–2115, Seattle, United States. Association for Computational Linguistics.

Ali Sial and mexwell. n.d. BreakingBad Script. Kaggle dataset. Accessed: 2026-04-14.

Studying Breaking Bad Course. n.d. Breaking Bad Episode Summaries. Episode summaries. Accessed: 2026-05-07.

SubsLikeScript. n.d. Breaking Bad (2008–2013): Episodes with Scripts. TV episode transcripts. Accessed: 2026-04-14.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*.